

Franck Pachot, dbi services

Star Transformation, 12c Adaptive Bitmap Pruning and In-Memory option

In the previous newsletters I've described Adaptive Plans, the 12c new feature where the CBO can generate multiple sub-plans and select the right one at the first execution time. And that was a pretext to describe join methods and parallel query distribution which are not always well known. But beside adaptive joins and adaptive parallel distribution, 12c comes with Adaptive Bitmap Pruning. So my articles become a trilogy and, as I did previously, I'll describe the case it applies to and which is often not well known: the Star Transformation.

Star schema

When a transactional application updates your data you store it in a structure that is close to what you insert: one table per business entity, and relational integrity among them. And you query usually in the same way, joining few rows from several tables.

But when a database is dedicated to query, and queries in BI often involve lot of rows, you prefer to store them close to the way you retrieve the data. You put all the measures that are related to same information (and same granularity) in a FACT table. And around that table with lot of rows you put smaller tables with all information about the axes of analysis – known as DIMENSIONS.

This is the star schema that I prefer to call a dimensional model.

A query on a star schema involves:

- Several predicates on dimension attributes (ex: sales date between two dates, country code in a list). They are columns on the dimension tables. Dimension tables are small (e.g. countries) or medium (e.g. customer)
- One or several measures to be retrieved. They are columns in the fact table, usually numbers. The fact table has a lot of rows and we usually need to read a lot of rows and aggregate them later
- Additional information from the dimension table (e.g. display the country name whereas predicate was on country code)

The usual access path (index range scan filtering all predicates and then access to table to get the measures) is not optimal or not possible for two reasons:

- Having all predicates in the same index is not possible because we can't have an index for each possible predicate combination
- Adding the additional information in the table would make it very large.

So basically, we build a schema with:

- One FACT table that has the minimum of columns (because it's already big because of the number of rows). Only the dimension key and the measures
- Several DIMENSION tables that has the key, columns where you will have predicates on, and all other information. They can be large (lot of columns) because they don't have a huge number of rows and can be denormalized (have all hierarchy) because they are quite static.
- The dimension keys in the FACT table are declared as foreign key to their DIMENSION tables.



Oracle Open World

Jedes Jahr wird die Berichterstattung der Oracle Openworld besser. Livestreams, Tickers usw. machen einen virtuellen Besuch möglich. Für jene die nicht unbedingt bis spät in die Nacht das Geschehen verfolgen wollten sind Aufzeichnungen aller Keynotes, garniert mit weiteren Informationen, verfügbar (<http://tinyurl.com/m2mpmq3>). Das funktioniert sogar mit dem Smartphone (mit dem der QR Code rechts besonders viel Spass macht):



- Each dimension key in the FACT table has a bitmap index on it so that all predicate results can be merge quickly before going to the large FACT table

Test case

I've build the following test case with one FACT table and three DIMENSION tables:

```
create table DIM1 as select rownum DIM1_ID , ... DIM1_COD, ... DIM1_TXT from dual connect by level<=10;
create table DIM2 as select DIM1_ID DIM2_ID,DIM1_COD DIM2_COD,DIM1_TXT DIM2_TXT from DIM1 where rownum<=10;
create table DIM3 as select DIM1_ID DIM3_ID,DIM1_COD DIM3_COD,DIM1_TXT DIM3_TXT from DIM1 where rownum<=10;
```

Those are my 3 dimension tables with an ID (the dimension key), a COD (where I'll have some criteria on) and a TXT (the additional information). I've 10 rows in each.

```
create table FACT as select rownum FACT_ID,DIM1_ID,DIM2_ID,DIM3_ID,mod(rownum,1000)/10 FACT_MESURE from
DIM1,DIM2,DIM3,(select * from dual connect by level<=1000);
```

This is my FACT table. I have 1000 rows per each combination of dimensions, so 1 million rows.

And I define the primary keys on the DIMENSION table and the foreign keys on the FACT table, as well as an index bitmap for each foreign key.

```
alter table DIM1 add constraint DIM1PK primary key(DIM1_ID);
alter table FACT add constraint DIM1FK foreign key (DIM1_ID) references DIM1;
create index DIM1BX on FACT(DIM1_ID);
alter table DIM2 add constraint DIM2PK primary key(DIM2_ID);
alter table FACT add constraint DIM2FK foreign key (DIM2_ID) references DIM2;
create index DIM2BX on FACT(DIM2_ID);
alter table DIM3 add constraint DIM3PK primary key(DIM3_ID);
alter table FACT add constraint DIM3FK foreign key (DIM3_ID) references DIM3;
create index DIM3BX on FACT(DIM3_ID);
```

Finally I gather statistics and, in order to simulate one large dimension, I fake the stats for DIM1 as if it has 100'000 rows:

```
exec dbms_stats.gather_schema_stats(user);
exec dbms_stats.set_table_stats(user,'DIM1',numrows=>1e5);
```

And now it's time to check some execution plans. I'm running the following query:

```
explain plan for select * from FACT
join DIM1 using(DIM1_ID)
join DIM2 using(DIM2_ID)
join DIM3 using(DIM3_ID)
where DIM1_COD='One' and DIM2_COD='One' and DIM3_COD='One';
```

That is:

- Predicate on DIM1, DIM2, DIM3 columns
- All measures from FACT
- Additional information from DIM1, DIM2, DIM3

Without star transformation

Here is the execution plan when I leave the 'star_transformation_enabled' to its default which is false:

NAME	TYPE	VALUE
star_transformation_enabled	string	FALSE

I use SQL Monitor which, in 12.1.0.2, shows adaptive plans, having the inactive part in gray:

Operation	Name	Exec...	Actual Rows
SELECT STATEMENT		1	1
HASH JOIN		1	1
NESTED LOOPS		1	1,000
NESTED LOOPS		1	1,000
STATISTICS COLLECTOR		1	1
MERGE JOIN CARTESIAN		1	1
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM3	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM1	1	1
BITMAP CONVERSION TO ROWIDS		1	1,000
BITMAP AND		1	1
BITMAP INDEX SINGLE VALUE	DIM1BX	1	6
BITMAP INDEX SINGLE VALUE	DIM2BX	1	12
BITMAP INDEX SINGLE VALUE	DIM3BX	1	40
TABLE ACCESS BY INDEX ROWID	FACT	1,000	1,000
TABLE ACCESS FULL	FACT	0	0

That's a long plan but not so complex. Here is what it does:

- First it reads all the dimensions DIM1, DIM2 and DIM3 (each one filtered with its own predicate) and does a cartesian join to get all the combination that are allowed by our predicates. This resultset has the dimension key to get to the FACT and has also the additional information we need for the final result.
- Then the STATISTICS COLLECTOR will decide on the sub-plan to choose (this is 12c adaptive join as I described in the previous newsletter).
- If the number of combination is not too large, it will do a NESTED LOOP: for each combination we get to the matching FACT rows. This is done through the bitmap indexes: for each dimension key, the corresponding bitmap index is accessed (BITMAP INDEX SINGLE VALUE), giving a bitmap of rows which are merge (BITMAP AND) and converted to ROWID. Then with those ROWID we loop to access to the FACT table.
- If the number of combination is large, then it is better to full scan the FACT table and do the join with the dimension combination through a HASH JOIN.

Star transformation without temporary table

Let's enable star transformation:

```
SQL> alter session set star_transformation_enabled=temp_disable;
Session altered.
```

Yes, there is no mistake here. Star transformation is enabled but without 'temp' which we will see later.

Do you remember that I've described star queries with two accesses to dimensions? One to apply the predicate (and get the dimension key for the result) and the other one to get the additional information once we got the FACT rows.

The principle of STAR transformation is to push the first one as if it were and IN (SELECT ID from DIM WHERE ...)

So let's look at the plan:

Operation	Name	Exec...	Actual Rows
SELECT STATEMENT		1	1,000
HASH JOIN		1	1,000
MERGE JOIN CARTESIAN		1	1
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM3	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM1	1	1
VIEW	VW_ST_A	1	1,000
NESTED LOOPS		1	1,000
BITMAP CONVERSION TO ROWIDS		1	1,000
BITMAP AND		1	1
BITMAP MERGE		1	1
BITMAP KEY ITERATION		1	6
TABLE ACCESS FULL	DIM1	1	1
BITMAP INDEX RANGE SCAN	DIM1BX	1	6
STATISTICS COLLECTOR		1	2
BITMAP MERGE		1	2
BITMAP KEY ITERATION		1	12
TABLE ACCESS FULL	DIM2	1	1
BITMAP INDEX RANGE SCAN	DIM2BX	1	12
STATISTICS COLLECTOR		1	5
BITMAP MERGE		1	5
BITMAP KEY ITERATION		1	40
TABLE ACCESS FULL	DIM3	1	1
BITMAP INDEX RANGE SCAN	DIM3BX	1	40
TABLE ACCESS BY USER ROWID	FACT	1,000	1,000

The first part – the MERGE JOIN CARTESIAN – is similar, but now the BITMAP INDEX SINGLE VALUE has been replaced. We read the dimension, apply the predicate, and for each dimension key we get to the bitmap index (BITMAP INDEX RANGE SCAN). The bitmaps are then merged for each dimension (BITMAP MERGE) and then ANDed with the ones coming from the other dimensions.

This is very efficient when the predicate has a good selectivity.

But what if the dimension is a big table? We have to read it two times here.

Star transformation with temporary table

Let's enable star transformation with temporary table:

```
SQL> alter session set star_transformation_enabled=true;
Session altered.
```

Remember, I've set the stats so that DIM1 appears as a large dimension. In order to avoid to read it two times, the optimizer can choose to put it in a temporary table. Look at the beginning of the plan:

The DIM1 table is first loaded as a temporary table. Then that temporary table is used in the dimension merge cartesian join (to build the hash table to lookup for the additional information) and is also used to apply the predicate before going to the DIM1BIX bitmap index:

Operation	Name	Ex...	Actual Rows
SELECT STATEMENT		1	1,000
TEMP TABLE TRANSFORMATION		1	1,000
LOAD AS SELECT		1	2
TABLE ACCESS FULL	DIM1	1	1
HASH JOIN		1	1,000
MERGE JOIN CARTESIAN		1	1
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	SYS_TEMP	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM3	1	1
VIEW	VW_ST_B7	1	1,000
NESTED LOOPS		1	1,000
BITMAP CONVERSION TO ROWIDS		1	1,000
BITMAP AND		1	1
BITMAP MERGE		1	1
BITMAP KEY ITERATION		1	6
TABLE ACCESS FULL	SYS_TEMP	1	1
BITMAP INDEX RANGE SCAN	DIM1BIX	1	6
STATISTICS COLLECTOR		1	2
BITMAP MERGE		1	2
BITMAP KEY ITERATION		1	12
TABLE ACCESS FULL	DIM2	1	1
BITMAP INDEX RANGE SCAN	DIM2BIX	1	12
STATISTICS COLLECTOR		1	5
BITMAP MERGE		1	5
BITMAP KEY ITERATION		1	40
TABLE ACCESS FULL	DIM3	1	1
BITMAP INDEX RANGE SCAN	DIM3BIX	1	40
TABLE ACCESS BY USER ROWID	FACT	1,000	1,000

Nothing else different. It's the same principle:

- Join each dimension to the FACT bitmap indexes
- Get the resulting ROWIDs and get the FACT rows
- Then join back to the dimension cartesian join in order to get additional information.

12c Adaptive Bitmap Pruning

So, all that exists before 12c. What is new is that grayed 'STATISTICS COLLECTOR'. I said that star transformation is good when the predicate is selective enough to filter few rows. Imagine that the cardinality estimation was wrong, and most of FACT rows have the required value. Then the optimizer can choose to stop iterating in that bitmap branch. We just ignore the predicate at that step, and the join back to the dimension Cartesian join will filter it anyway.

If you check the execution plan with predicates, you will see the predicate on dimension in the two table access.

Here I still run the same query but I've changed my data. In the previous examples, only 1 row was coming out from the DIM3 dimension (Actual Rows in the execution plan). Now I have 6 rows in DIM3 that are returned:

Operation	Name	Ex...	Actual Rows
SELECT STATEMENT		1	6,000
TEMP TABLE TRANSFORMATION		1	6,000
LOAD AS SELECT		1	2
TABLE ACCESS FULL	DIM1	1	1
HASH JOIN		1	6,000
MERGE JOIN CARTESIAN		1	6
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	SYS_TEMP	1	1
BUFFER SORT		1	6
TABLE ACCESS FULL	DIM3	1	6
VIEW	VW_ST_B7	1	6,000
NESTED LOOPS		1	6,000
BITMAP CONVERSION TO ROWIDS		1	6,000
BITMAP AND		1	1
BITMAP MERGE		1	1
BITMAP KEY ITERATION		1	6
TABLE ACCESS FULL	SYS_TEMP	1	1
BITMAP INDEX RANGE SCAN	DIM1BIX	1	6
STATISTICS COLLECTOR		1	2
BITMAP MERGE		1	2
BITMAP KEY ITERATION		1	12
TABLE ACCESS FULL	DIM2	1	1
BITMAP INDEX RANGE SCAN	DIM2BIX	1	12
STATISTICS COLLECTOR		1	5
BITMAP MERGE		1	5
BITMAP KEY ITERATION		1	240
TABLE ACCESS FULL	DIM3	1	6
BITMAP INDEX RANGE SCAN	DIM3BIX	6	240
TABLE ACCESS BY USER ROWID	FACT	6,000	6,000

Look at the end. When the statistics collector has seen that the threshold has been passed over, it has decided to skip that bitmap branch. This is the third case of adaptive plans: Adaptive Bitmap Pruning. The bitmap branch is good only if it helps to filter a lot of rows. If it's not the case, then it's just an overhead, and it is skipped coming back to the behavior we had at the beginning when star transformation was disabled.

12c In-Memory option

I'm talking about new features, and you probably are tired of long execution plans. So let's try In-Memory:

```
alter table FACT inmemory priority critical;
alter table DIM1 inmemory priority critical;
alter table DIM2 inmemory priority critical;
alter table DIM3 inmemory priority critical;
```

I've plenty of memory and I've set my inmemory_size already. I just have to wait a bit, so that the in-memory column store is filled and run my query again:

Operation	Name	Id...	Ex...	Act...	Timeline(...)	Me...
SELECT STATEMENT		0	1	1,000		
HASH JOIN		1	1	1,000		288KB
JOIN FILTER CREATE	:BF0000	2	1	1		
MERGE JOIN CARTESIAN		3	1	1		
MERGE JOIN CARTESIAN		4	1	1		
TABLE ACCESS INMEMORY FULL	DIM2	5	1	1		
BUFFER SORT		6	1	1		2KB
TABLE ACCESS INMEMORY FULL	DIM3	7	1	1		
BUFFER SORT		8	1	1		2KB
TABLE ACCESS INMEMORY FULL	DIM1	9	1	1		
JOIN FILTER USE	:BF0000	10	1	100K		
TABLE ACCESS INMEMORY FULL	FACT	11	1	100K		

Of course, you need to have the FACT table in memory, or at least the interesting partition. But then you don't need star transformation. As usual you have that cartesian merge join to get the dimension. But then you remember that, without star transformation, you accessed to the FACT through NESTED LOOP JOIN or HASH JOIN – and that was adaptive.

Here the FACT is stored in memory and there is no index access, so we use a full scan. Do you remember that star transformation was nice because it pushes down the predicates to filter the FACT table earlier? Here we have something else. The criteria are pushed with a bloom filter. Because we have read all the dimensions first, then we can build the

bloom filter (JOIN FILTER CREATE) and use it (JOIN FILTER USE) to filter a large part of the rows – saving the cost of lots of hash lookups. Hash lookups have to be done only for the few bloom filter false positives. And vector processing, which is the way to scan columnar data, is very efficient with bloom filters.

Conclusion

This completes the trilogy about adaptive plans that appeared in 12c. I'm sure that a refresh about star transformation was not a bad idea. I've worked a lot on datawarehouses and star schemas but still had to study it when preparing the OCM exam. And the adaptive feature in this area has not been widely documented. I've concluded with In-Memory because I think that the star transformation, and especially the bitmap indexes, was a premise of a columnar approach. The problem is that they don't like OLTP updates. You can have star transformation with regular indexes as well, but there is a rowid-to-bitmap transformation that has a big overhead. In-Memory is a good solution for ad-hoc queries on OLTP databases – as long as you have enough memory to keep your data in the in-memory column store. But about that, keep in mind that columnar compression has great ratios on fact tables because of the repeated dimension keys. ■

Contact

dbi services

Franck Pachot

E-Mail:

franck.pachot@dbi-services.com

SMS

Neue Oracle Cloud Services

Oracle hat die Verfolgung der etablierten Cloud Service Provider definitiv aufgenommen und addiert sechs neue Dienste:

- Oracle Big Data cloud – nicht ganz die erste Cloud Lösung eines Hadoop frameworks
- Oracle Mobile Cloud für «Enterprise-grade» Mobile Apps...
- Oracle Integration Cloud verschmelzt Cloud/Cloud und Cloud/OnPrem über ein BUI
- Oracle Process Cloud ermöglicht das einfache Abbilden von Geschäftsprozessen. Oder so.
- Oracle Node.js Cloud für Java Scripter in der Wolke
- Oracle JAVE SE Cloud für JAVA SE Anwendungen

Zusammen mit den bereits verfügbaren Diensten steht somit einem Geschäft eine umfangreiche Sammlung von Möglichkeiten und Diensten zur Verfügung. Mit den breit verfügbaren Trial Möglichkeiten und Preisplänen lohnt sich eine genauere Betrachtung. Wir empfehlen allerdings für diese genaue Betrachtung auch wirklich Zeit zu investieren – an einem Abend ist das nicht erledigt.

